Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

# Raport
## la
## Lucrarea de laborator Nr. 10

*Disciplina: Tehnici de programare*

**Tema:** "Funcții și excepții"

A efectuat : st. gr. SI–262
Inculeț Andrian

A verificat : asistent universitar
Dumitrașcu Marius

Chișinău – 2026

# SARCINA LUCRĂRII

## LUCRAREA A (nucleu). parse_auth.py: scoate și numără eșecurile

Obiectiv: arsenal/aparare/parse_auth.py citește jurnalul, scoate eșecurile de autentificare într-o listă de fișe și le numără pe adrese. Robust: un rând stricat nu oprește parcurgerea.

### A1. Scheletul și funcția care întoarce o fișă (20 min)

Un rând de eșec arată așa: Failed password for root from 203.0.113.9 port 51230 ssh2. Scrie o funcție esec(linie) care întoarce (user, ip) dacă rândul e un eșec, altfel None. Folosești o expresie regulată dată:

```
import re
TIPAR = re.compile(r"Failed password for (?:invalid user )?(\S+) from (\S+) port")

def esec(linie):
    m = TIPAR.search(linie)
    # TODO: daca m e None, intoarce None;
    #       altfel intoarce (m.group(1), m.group(2))
    ...

print(esec("... Failed password for root from 203.0.113.9 port 51230 ssh2"))
print(esec("<linie stricata fara structura>"))
```

**Pe ecran trebuie să vezi:**

```
('root', '203.0.113.9')
None
```

**De observat.** Funcția întoarce None pentru un rând care nu e un eșec. None nu e o eroare, e un semnal: „aici nu e nimic de raportat". Cel care cheamă funcția decide ce face cu el.

### A2. Lista de eșecuri, robustă la rânduri stricate (20 min)

citeste_esecuri(cale) parcurge fișierul și strânge fișele (user, ip) într-o listă de dicționare. Sare peste rândurile care nu sunt eșecuri. Pune citirea fișierului într-un try/except, ca un fișier lipsă să dea un mesaj clar, nu un traceback.

```
def citeste_esecuri(cale):
    esecuri = []
    try:
        f = open(cale, encoding="utf-8")
    except FileNotFoundError:
        print("nu gasesc", cale)
        return esecuri
    with f:
        for linie in f:
            r = esec(linie)
            # TODO: daca r nu e None, adauga {"user": r[0], "ip": r[1]}
    return esecuri

esecuri = citeste_esecuri("arsenal/aparare/start/auth.log")
print("esecuri:", len(esecuri))
```

**Pe ecran trebuie să vezi:**

```
esecuri: 15
```

## A3. Numărarea pe IP: o funcție cu return (18 min)

Numeri de câte ori apare fiecare adresă. Poți cu un dicionar și .get, sau cu Counter din collections, care face exact asta. Funcția ÎNTOARCE numărătoarea, nu o tipărește.

```
from collections import Counter

def pe_ip(esecuri):
    # TODO: intoarce un Counter al adreselor din esecuri
    #       (Counter(e["ip"] for e in esecuri))
    ...

counts = pe_ip(esecuri)
for ip, n in counts.most_common():
    print(f"  {ip}: {n}")
```

**Pe ecran trebuie să vezi:**

```
  203.0.113.9: 13
  198.51.100.7: 2
```

**De observat.** O adresă iese mult peste rest: 13 eșecuri față de 2. Asta e semnătura unui brute force. most_common() sortează de la mare la mic, deci atacatorul apare primul.

## A4. Rezumatul (12 min)

O funcție main() leagă totul: citește, numără, tipărește rezumatul. Rulezi din rădăcina depozitului.

```
python -m arsenal.aparare.parse_auth arsenal/aparare/start/auth.log
```

**Pe ecran trebuie să vezi:**

```
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
```

> **DE REȚINUT.** Fiecare funcție întoarce un rezultat (fișa, lista, numărătoarea). Le legi ca pe o conductă: ieșirea uneia intră în următoarea. O funcție care doar tipărea ar fi rupt lanțul.

```
import argparse
import re
from collections import Counter

TIPAR = re.compile(r"Failed password for (?:invalid user )?(\S+) from (\S+) port")

def esec(linie):
    m = TIPAR.search(linie)

    if m is None:
        return None

    return m.group(1), m.group(2)

def citeste_esecuri(cale):
    esecuri = []
    try:
        f = open(cale, encoding="utf-8")
    except FileNotFoundError:
```

```
        print("nu gasesc", cale)
        return esecuri
    with f:
        for linie in f:
            r = esec(linie)

            if r is not None:
                esecuri.append({
                    "user": r[0],
                    "ip": r[1]
                })
    return esecuri

def pe_ip(esecuri):
    return Counter(e["ip"] for e in esecuri)

def main():
    p = argparse.ArgumentParser()
    p.add_argument("auth")
    a = p.parse_args()

    auth_path = a.auth

    print("\n\nA4")
    esecuri = citeste_esecuri(auth_path)
    print("Esecuri de autentificare:", len(esecuri))
    counts = pe_ip(esecuri)
    for ip, n in counts.most_common():
        print(f"  {ip}: {n}")

if __name__ == "__main__":

    print("\n\nA1")
    print(esec("... Failed password for root from 203.0.113.9 port 51230 ssh2"))
    print(esec("<linie stricata fara structura>"))

    print("\n\nA2")
    esecuri = citeste_esecuri("./arsenal/aparare/start/auth.log")
    print("esecuri:", len(esecuri))

    print("\n\nA3")
    counts = pe_ip(esecuri)
    for ip, n in counts.most_common():
        print(f"{ip}: {n}")
    main()
```

Output la partea A – Fig1.1 – python3 arsenal/aparare/parse_auth.py ./arsenal/aparare/start/auth.log

```
A1
('root', '203.0.113.9')
None


A2
esecuri: 15
```

```
A3
203.0.113.9: 13
198.51.100.7: 2


A4
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
```

```
(.venv) xperiencez@lenovo-v15-g4-iru:~/Documents/GitHub/tehnici-de-programare$ python3 arsenal/aparare/parse_auth.py ./arsenal/aparare/start/auth.log


A1
('root', '203.0.113.9')
None


A2
esecuri: 15


A3
203.0.113.9: 13
198.51.100.7: 2


A4
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
```

## B2. Validarea, verificată (8 min)

Verifică faptul că un prag absurd e oprit cu un mesaj clar:

```
python -c "from arsenal.aparare.detector import suspecti; suspecti({}, 0)"
```

**Pe ecran trebuie să vezi:**

```
ValueError: prag invalid: 0 (cel putin 1)
```

**De observat.** raise oprește funcția la intrare, cu o explicație. E mai bine decât să continue și să marcheze orice adresă ca suspectă, în tăcere.

## B3. argparse și raportul JSON (22 min)

main() primește jurnalul, un --prag (implicit 10, type=int) și un --json. Leagă funcțiile din parse_auth cu suspecti și scoate fie un rezumat citibil, fie un raport JSON.

```
import argparse, json
from collections import Counter
from arsenal.aparare.parse_auth import citeste_esecuri

def main():
    p = argparse.ArgumentParser()
    p.add_argument("log")
    p.add_argument("--prag", type=int, default=10)
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    esecuri = citeste_esecuri(a.log)
    counts = Counter(e["ip"] for e in esecuri)
    sus = suspecti(counts, a.prag)
    # TODO: cu --json, scoate json.dumps cu {esecuri, pe_ip, suspecti};
```

```
    #      fara, un rezumat cu "BRUTE FORCE" pe fiecare suspect
    ...
python -m arsenal.aparare.detector arsenal/aparare/start/auth.log
```

**Pe ecran trebuie să vezi:**

```
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
Suspecti (prag 10):
  203.0.113.9  (13 incercari)  BRUTE FORCE
```

### B4. Un test care prinde regresia (15 min)

Scrie arsenal/aparare/test_detector.py cu două teste cu date scrise de mână: numărarea e corectă și pragul taie unde trebuie. Rulează fără jurnal real.

```
from collections import Counter
from arsenal.aparare.detector import suspecti

def test_pragul_taie():
    c = Counter({"a": 13, "b": 2})
    assert suspecti(c, 10) == [("a", 13)]

def test_prag_invalid():
    import pytest
    with pytest.raises(ValueError):
        suspecti({}, 0)
pytest -q arsenal/aparare/test_detector.py
```

**Pe ecran trebuie să vezi:**

```
2 passed
```

```
import argparse
import json

from arsenal.aparare.parse_auth import citeste_esecuri, pe_ip
from collections import Counter

def suspecti(counts, prag=10):
    if prag < 1:
        raise ValueError(f"prag invalid: {prag} (cel putin 1)")

    rezultat = [(ip, n) for ip, n in counts.items() if n >= prag]
    rezultat.sort(key=lambda x: x[1], reverse=True)
    return rezultat



def main():
    p = argparse.ArgumentParser()
    p.add_argument("log")
    p.add_argument("--prag", type=int, default=10)
    p.add_argument("--json", action="store_true")
```

```
    a = p.parse_args()

    esecuri = citeste_esecuri(a.log)
    counts = Counter(e["ip"] for e in esecuri)
    sus = suspecti(counts, a.prag)

    if a.json:
        raport = {
            "esecuri": esecuri,
            "pe_ip": dict(counts),
            "suspecti": sus,
        }
        print(json.dumps(raport, indent=2))
    else:
        print(f"\n\nEsecuri de autentificare: {len(esecuri)}")
        for ip, n in counts.most_common():
            print(f"{ip}: {n}")

        print(f"\n\nSuspecti (prag {a.prag}):")
        for ip, n in sus:
            print(f"{ip} ({n} incercari) BRUTE FORCE")


if __name__ == "__main__":
    c = Counter({"203.0.113.9": 13, "198.51.100.7": 2})
    print(suspecti(c))
    print(suspecti(c, prag=2))

    main()
```

Output la partea B – Fig. 2 - python -m arsenal.aparare.detector arsenal/aparare/start/auth.log

```
A1
('root', '203.0.113.9')
None


A2
esecuri: 15


A3
203.0.113.9: 13
198.51.100.7: 2


A4
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
```

```
(.venv) xperiencez@lenovo-v15-g4-iru:~/Documents/GitHub/tehnici-de-programare$ python -m arsenal.aparare.parse_auth arsenal/aparare/start/auth.log


A1
('root', '203.0.113.9')
None


A2
esecuri: 15


A3
203.0.113.9: 13
198.51.100.7: 2


A4
Esecuri de autentificare: 15
  203.0.113.9: 13
  198.51.100.7: 2
```

```
from collections import Counter
from arsenal.aparare.detector import suspecti


def test_pragul_taie():
    c = Counter({"a": 13, "b": 2})
    assert suspecti(c, 10) == [("a", 13)]


def test_prag_invalid():
    import pytest
    with pytest.raises(ValueError):
        suspecti({}, 0)
```

Output la test – Fig. 3 - pytest -q arsenal/aparare/test_detector.py

2 passed in 0.01s

```
(.venv) xperiencez@lenovo-v15-g4-iru:~/Documents/GitHub/tehnici-de-programare$ pytest -q arsenal/aparare/test_detector.py
..
2 passed in 0.01s
```

## Concluzie

In urma realizarii acestei lucrari de laborator, am invatat cum sa folosesc functiile si exceptiile in limbajul Python. Am creat functii care citesc informatii din fisiere, identifica incercarile nereusite de autentificare si numara adresele IP de la care au fost efectuate acestea.

Pe langa asta, am invatat cum sa tratez erorile folosind try except, cum sa verific valorile introduse cu raise si cum sa creez rapoarte Json. Am facut si teste automate, toate cele doua teste fiind executate cu succes.

In concluzie, aceasta lucrare m-a ajutat sa inteleg mai bine cum functioneaza functiile, cum pot fi gasite si tratate erorile si cat de important este sa scriem cod organizat si corect.